

SaaS Context Broker

Project analysis and codebase context report

This project is a Spring Boot incident-broker backend that ingests Slack and Jira updates, stores incident state in PostgreSQL, computes risk and severity, emits timeline events, and auto-escalates unresolved critical incidents.

Generated: 2026-06-11 10:48

Stack: Java 21, Spring Boot 4.0.2, Spring Data JPA, PostgreSQL

1. What this project is

The application is a backend-only SaaS context broker for incidents. It sits between external event sources and an operational dashboard, normalizing incoming Slack messages and Jira state changes into a single incident model. The core goal is to keep incident context, risk scoring, status transitions, timeline history, and alerting in one service.

The repo currently looks like an early-stage but coherent portfolio project: the business flow is implemented, the API surface is broad, and the architecture is split into controllers, services, entities, repositories, DTOs, enums, and exception handling.

2. How it works

Stage	Behavior
Incoming Slack	POST /incoming/slack creates or updates an incident from message text.
Incoming Jira	POST /incoming/jira syncs lifecycle state from Jira status changes.
Incident service	Computes severity, risk score, alert eligibility, and timeline events.
Persistence	Incident state, timeline events, and Slack webhook config are stored in PostgreSQL.
Notifier	High-severity incidents send Slack alerts using the configured webhook.
Monitor	A scheduled job re-checks incidents and escalates stale critical ones.

The design is event-led rather than UI-led: external systems feed the service, the service keeps a canonical incident record, and analytics endpoints expose state for a frontend dashboard.

3. What it can do today

Capability	Evidence in code
Slack ingest	SlackController + SlackIncomingRequest + IncidentService.handleUpdate
Jira ingest	JiraaController + JiraIncomingRequest + IncidentService.handleJiraUpdate
Risk/severity scoring	IncidentService.evaluateSeverity and calculateRiskScore
Metrics dashboard	GET /incident/metrics, /system-health, /trend, /severity-distribution
Incident lookup	GET /incident/{incidentKey} and /incident/incident-details/{incidentKey}
Incident assignment	POST /incident/{incidentKey}/assign
Audit timeline	TimelineService + GET /incident/{incidentKey}/timeline
Slack escalation	SlackNotifier + IncidentMonitor
Webhook configuration	POST /config/slack

The strongest current capability is the end-to-end incident lifecycle: ingest, classify, track, notify, and query. It already behaves like a small incident operations backend rather than a simple webhook collector.

4. Core architecture

Layer	Role
Controller	HTTP API entrypoints and request validation
Service	Incident rules, scoring, notification, timeline orchestration
Repository	JPA persistence access

Layer	Role
Entity	Domain state stored in PostgreSQL
DTO	Request/response shapes for API payloads
Model	Enums for severity and lifecycle status
Config/Exception	CORS setup and structured error handling

The split is clean and understandable. The only architectural smell is that some logic is duplicated across scoring paths, and some historical models still exist alongside the new unified timeline model.

5. Important behavior details

- Scoring weights are configurable in application.properties: urgent=50, jiraOpen=30, jiraInProgress=10, stale=20.
- Severity tiers are LOW, MEDIUM, HIGH, CRITICAL.
- Resolved and closed incidents are treated as terminal and get risk score 0 in read APIs.
- Timeline data is merged from the new TimelineEventEntity and the legacy IncidentEventEntity source.
- CORS is currently hardcoded for http://localhost:3000.
- The app uses PostgreSQL with ddl-auto=update and actuator dependency is present.

6. Current gaps and risks

Area	Issue
Consistency	Risk scoring is implemented in more than one way; the read score and the alert score do not use identical
Monitoring	IncidentMonitor uses a hardcoded 1-minute stale threshold instead of the configured threshold.
Metrics	escalationsToday is always 0, so the metric is stubbed rather than implemented.
Reliability	Slack notifications use RestTemplate without retry, timeout, or failure handling.
Security	No authentication, authorization, or webhook signature verification is visible.
Config	Database credentials and allowed frontend origin are hardcoded.
Testing	Only the Spring context-load test exists.
Data model	WebhookConfig behaves like a singleton table without explicit enforcement.
API quality	JiraaController name has a typo and endpoint naming is slightly inconsistent.

7. What should be improved next

Priority	Improvement
High	Unify scoring into one source of truth so alerting, API reads, and monitoring all agree.
High	Externalize hardcoded cooldowns, monitor cadence, and CORS settings into configuration.
High	Add authn/authz for operational endpoints and validate incoming webhook authenticity.
High	Replace system prints and silent webhook fallbacks with structured logging and explicit error paths.
Medium	Add unit tests for scoring, assignment, stale detection, and controller validation.
Medium	Add OpenAPI/Swagger docs so the API surface is discoverable.
Medium	Model users or assignees instead of plain strings if assignment becomes real workflow state.
Medium	Remove or formally migrate legacy timeline persistence once historical compatibility is no longer needed.

Priority	Improvement
Low	Polish naming, comments, and DTO duplication.

8. Features that appear missing

- Authentication and role-based access control.
- Webhook signature verification and source trust validation.
- Notification retries, backoff, and failure tracking.
- A real concept of users/teams/ownership instead of free-text assignment.
- Search/sort capabilities for incidents beyond the current filters.
- Historical reporting beyond the existing 7-day trend and simple metrics.
- Production config management and secret storage.

9. File references that matter most

- pom.xml: Spring Boot 4.0.2, JPA, validation, actuator, PostgreSQL.
- application.properties: database connection and scoring parameters.
- SaaSContextBrokerApplication.java: application bootstrap and scheduling enabled.
- IncidentService.java: core scoring, alerting, metrics, priority, and trend logic.
- IncidentController.java: main read API surface.
- SlackController.java, JiraaController.java: inbound webhook ingestion.
- TimelineService.java: audit/event history.
- IncidentMonitor.java: scheduled background escalation.
- GlobalExceptionHandler.java: structured error responses.

10. Bottom line

This is a solid incident-intelligence backend in an early product stage. It already demonstrates useful backend engineering: layered Spring structure, persistence, workflow orchestration, scoring logic, timeline tracking, and automated escalation. The biggest value now is to harden the consistency, security, and observability so it can move from a promising prototype into a dependable service.